Innovation Centre for Education

Low-Level Design

for

Hardened Cloud Infrastructure for Retail Data

Mohammed Saqib

23BCCE117

BCA in Cloud Computing, Cyber Security & Ethical Hacking with IBM

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

11-04-2026

Under the guidance of

Ms. Munisha

Lecturer, Computer Science Department
Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

Submitted to

YENEPOYA
(DEEMED TO BE UNIVERSITY)
Recognized under Sec 3(A) of the UGC Act 1956
Accredited by NAAC with 'A' Grade

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

BALMATTA, MANGALORE

YENEPOYA (DEEMED TO BE UNIVERSITY)

Table of Content

1. Introduction
   1.1 Scope of the Document
   1.2 Intended Audience
   1.3 System Overview
2. System Design
   2.1 Application Architecture Diagram
   2.2 Process Flow (Dispatch Lifecycle) – Sequence Diagram
   2.3 Information Flow (Telemetry Engine) – Data Flow Diagram
   2.4 Components Design (Detailed)
   2.5 Key Design Considerations
   2.6 API Catalogue
3. Data Design
   3.1 Entity-Relationship (ER) Diagram
   3.2 Data Model (Schema & Encryption)
   3.3 Data Access Mechanism
   3.4 Data Retention & Archival Policies
   3.5 Data Migration & Seeding
4. Interfaces
   4.1 User Interface (UI) Layout
   4.2 API Contracts (Request/Response Structure)
5. State and Session Management
6. Caching Strategy
7. Non-Functional Requirements
   7.1 Security Aspects
   7.2 Performance Aspects
8. Conclusion

1. Introduction

1.1 Scope of the Document

This Low-Level Design (LLD) document provides detailed technical specifications for implementing the Layered Retail Cloud Security framework. It covers Azure infrastructure provisioning (VNet, subnets, NSG, Application Gateway with WAF_v2), the Python-based encryption engine (Argon2id key derivation, AES-256-GCM authenticated encryption), Flask API endpoints for secure file upload/download, database schema for metadata and audit logs, session management, caching strategy, and non-functional requirements (security + performance). The document serves as the complete implementation blueprint for DevOps engineers and Python developers.

1.2 Intended Audience

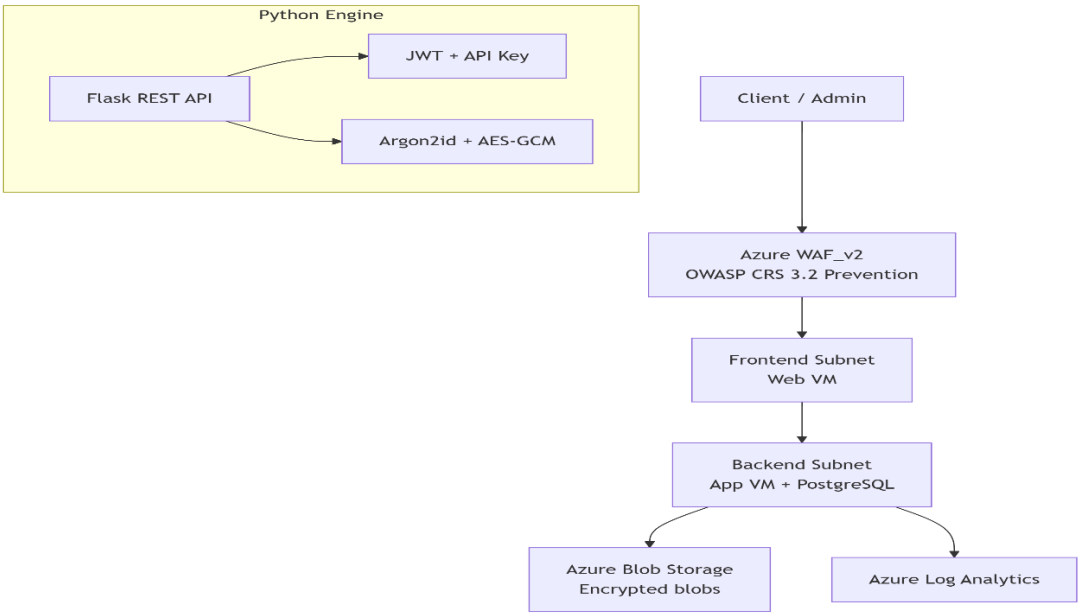
- **Cloud Engineers –** Provision Azure resources using Terraform.
- **Developers –** Implement the Python encryption engine and REST API.
- **Security Testers –** Validate WAF rules, NSG isolation, and cryptographic correctness.
- **Retail IT Administrators –** Understand the defense-in-depth controls.

1.3 System Overview

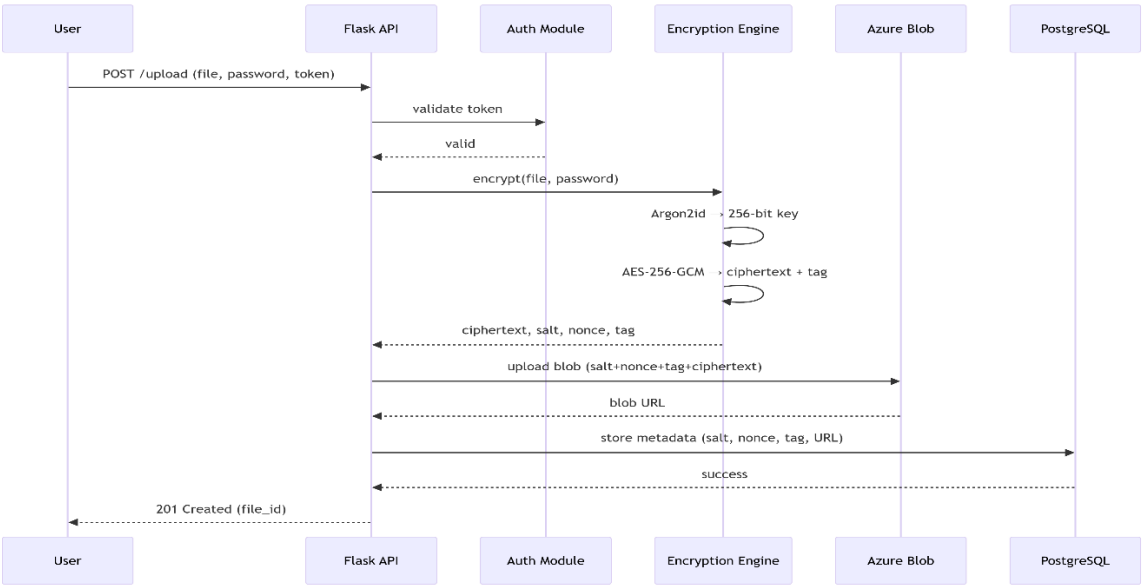
Flask-based web application on Microsoft Azure that implements defense-in-depth for retail data. Azure WAF_v2 (OWASP CRS 3.2) blocks SQLi/XSS at the perimeter. A VNet with frontend and backend subnets isolates the application VM and database; NSG rules allow backend access only from the frontend. Data-in-transit uses TLS 1.3. For data-at-rest, a Python engine derives a 256-bit key via Argon2id (64MB memory, 3 iterations) and encrypts files using AES-256-GCM. The ciphertext (with salt, nonce, tag) goes to Azure Blob Storage; metadata to PostgreSQL. Access is controlled via stateless JWT tokens with RBAC (Admin/User/Auditor). All events log to Azure Log Analytics. Even if the perimeter is breached, retail data stays encrypted.

2. System Design

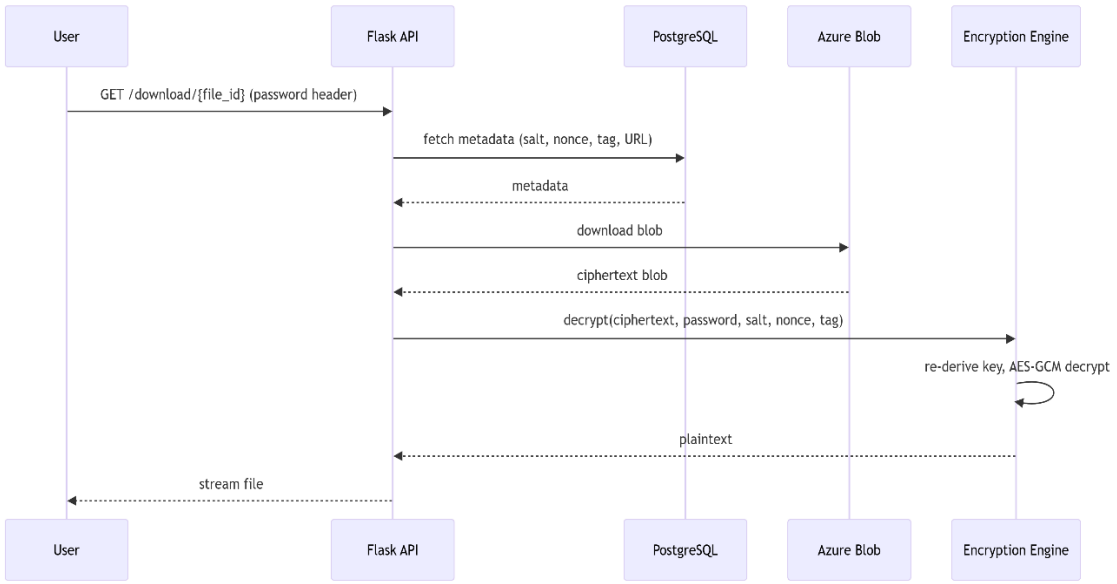
2.1 Application Design



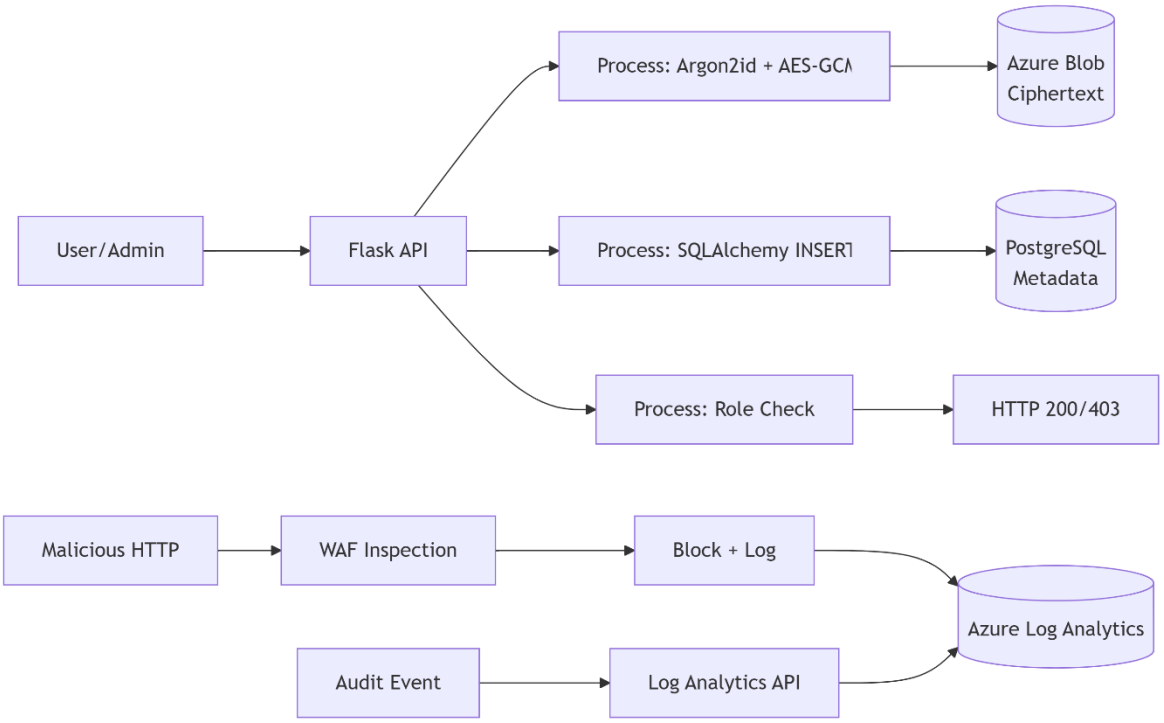
2.2 Process Flow (Dispatch Lifecycle) – Sequence Diagram:
Upload (Encryption) Sequence:



Download (Decryption) Sequence:



2.3 Information Flow (Telemetry Engine) – Data Flow Diagram



2.4 Components Design

Component	Technology	Specifications
Key Derivation	Argon2id (`argon2-cffi`)	Memory=64 MB, Time=3, Parallelism=1, Salt=16 B
Symmetric Cipher	AES-256-GCM (`cryptography`)	Nonce=12 B, Tag=16 B
Binary File Structure	Custom	`[Salt(16)][Nonce(12)][Tag(16)][Ciphertext]`
Infrastructure	Terraform 1.4+, Azure CLI	VNet `10.0.0.0/16`, subnets: frontend `10.0.1.0/24`, backend `10.0.2.0/24`
NSG Rule (Backend)	Azure NSG	Allow TCP 80,443 from `10.0.1.0/24` only
WAF	Azure WAF_v2	OWASP CRS 3.2, Prevention mode, log all triggers

2.5 Key Design Considerations

- **Defense-in-Depth – No single layer is trusted; overlapping controls.**
- **Zero-Trust Data – Encrypted before storage; cloud provider never sees plaintext.**
- **Performance – Argon2id memory cost fits within 2 GB VM.**
- **Compliance – AES-256-GCM meets FIPS 140-2 for data-at-rest.**

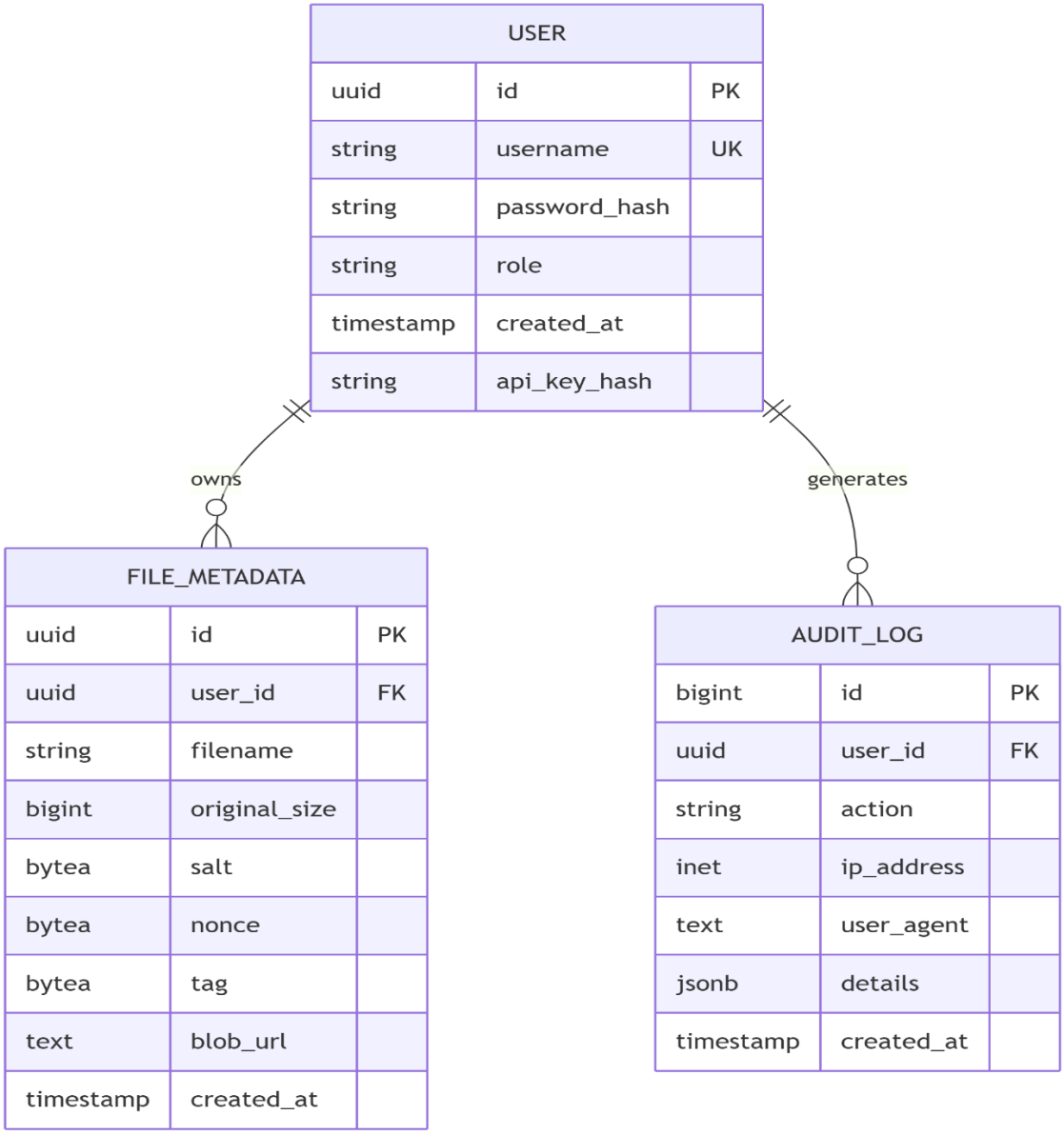
2.6 API Catalogue

Endpoint	Method	Description	Auth
`/upload`	POST	Upload and encrypt a file	JWT or API Key
`/download/{file_id}`	GET	Download and decrypt a file	JWT or API Key + password header
`/health`	GET	Health check	None

`/audit`	GET	Retrieve audit logs (admin only)	JWT (admin role)
`/login`	POST	Obtain JWT token	None

3. Data Design

3.1 Entity-Relationship (ER) Diagram



3.2 Data Model (Schema & Encryption)

- **Table users –** stores authentication data (passwords hashed with PBKDF2-SHA256).
- **Table file_metadata –** stores encryption parameters (salt, nonce, tag) and blob URL. Actual ciphertext is in Azure Blob.
- **Table audit_log –** logs all security events.
- **No plaintext or encryption keys are ever stored** – keys are derived ephemerally from user password

3.3 Data Access Mechanism

- **ORM:** SQLAlchemy with connection pooling.
- **Queries:** User.query.filter_by(username=username).first()
- **Transactions:** db.session.add() and commit().
- **Blob operations:** Azure Blob SDK upload_blob / download_blob.

3.4 Data Retention & Archival Policies

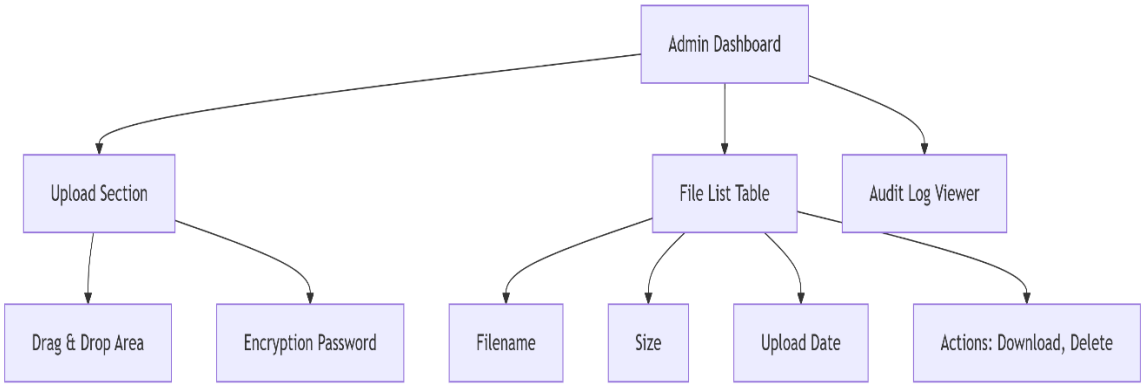
- **User accounts**: Permanent until deletion request.
- **File metadata & blobs:** Deleted when user account is removed.
- **Audit logs:** 180 days retention (Azure Log Analytics).
- **Archival:** Files >90 days old moved to Azure Cool Blob tier.

3.5 Data Migration & Seeding

- **Migration tool:** Alembic (Flask-Migrate).
- **Seed data:** 100 sample retail files pre-encrypted for testing.
- **Rollback:** Database snapshots + Alembic downgrade.

4. Interfaces

4.1 User Interface (UI) Layout



4.2 API Contracts (WebSocket Events)

POST /upload

Item	Description
Request Format	multipart/form-data
Request Fields	- file: binary file content - password: user-supplied string for key derivation
Response (Success – 201 Created)	file_id (UUID) and message ("File encrypted and stored")
Response (Failure)	Appropriate HTTP error code (400, 401, 413, etc.)

GET /download/{file_id}

Item	Description
Request Headers	`X-User-Password`: password string for decryption
Response (Success – 200 OK)	Binary file stream (original plaintext file)
Response (401)	Wrong password
Response (404)	File not found

POST /login

Item	Description
Request Body	`username` and `password` (plain text)
Response (200 OK)	`access_token` (JWT, expires 15 min), `refresh_token` (cookie)
Response (401)	Invalid credentials

GET /audit (admin only)

Item	Description
Request Header	`Authorization: Bearer <admin_jwt>`
Response (200 OK)	JSON array of audit log entries (action, timestamp, user, ip, details)
Response (403)	Insufficient role (non-admin)

5. State and Session Management

- **Authentication:** Stateless JWT (15 min expiry) + refresh token (7 days, HTTP-only cookie).
- **No server-side session store –** horizontal scaling ready.
- **Logout:** Client discards token; refresh cookie cleared.
- **Secure flags:** Refresh token cookie HttpOnly, Secure, SameSite=Strict.

6. Caching

- **Redis (**Azure Cache for Redis)
- **Cached item:** Derived encryption key derived_key:{hash(password+salt)} – TTL 5 minutes.
- **Cache invalidation:** On password change, delete all keys for that user.
- **No caching of plaintext –** security requirement.
- **Static assets**: Browser caching only.

7. Non-Functional Requirements

7.1 Security Aspects

Aspect	Implementation
Password storage (API auth)	PBKDF2-SHA256 (Werkzeug)
Data-at-rest encryption	AES-256-GCM + Argon2id
Data-in-transit	TLS 1.3 (Azure Gateway)
WAF	OWASP CRS 3.2 Prevention
Network isolation	NSG allow only frontend subnet
SQL injection	SQLAlchemy ORM
XSS	Jinja2 auto-escape (if UI added)
CSRF	Stateless JWT avoids CSRF
Audit logging	Azure Log Analytics

7.2 Performance Aspects

Metric	Target	Means
Encryption throughput	≥ 50 MB/s	AES-NI hardware acceleration
Argon2id latency	≤ 300 ms	64 MB memory, 3 iterations
API response (p95)	≤ 500 ms (excl. upload)	Redis cache, connection pooling
Concurrent users	100	Horizontal scaling
Uptime	99.9%	Azure availability zones + health probes

Conclusion

The Layered Retail Cloud Security LLD provides a complete technical blueprint for a defense-in-depth system. It combines Azure network security (WAF, NSG, VNet isolation) with a strong cryptographic engine (Argon2id + AES-256-GCM). The design ensures that even if the network perimeter is breached, retail data remains encrypted and inaccessible. All components are developer-ready, scalable, and compliant with modern security standards.